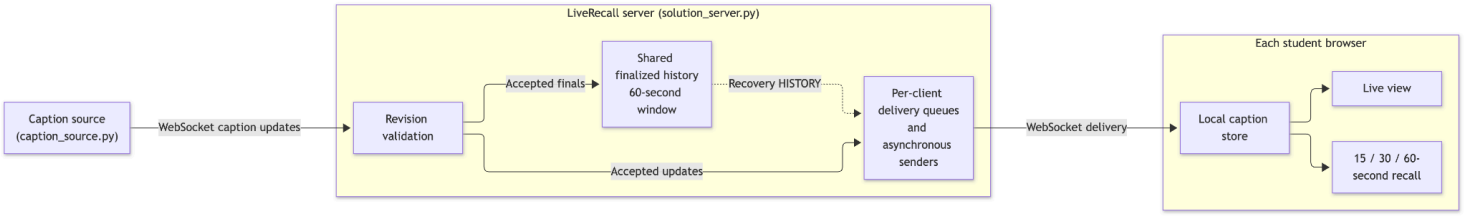
CS3103 Assignment 4 Technical Report

Team 16 - Ong Jun Yi (A0301890J), Julian Lim (A0292058Y), Xia Wenhao (A0252901M)
Repository URL: https://github.com/ojunyi/cs3103-liverecall

1 System Architecture and Protocol

The application consists of three components: the caption source, the LiveRecall server, and student browser clients.



These three components only communicate with each other over WebSocket, which itself runs on top of TCP. The source has a transcript and sends each caption as a series of revisions until it is finalized. Upon receiving a caption, the server checks for duplication and stale provisional segments (i.e., drop revision 2 if revision 3 has already been received before), and broadcasts a copy of each caption update to every client's own queue (located on the server). Depending on that client's state (e.g., lagging or normal), the update is handled differently, which will be described in more detail in later sections. The server also keeps a history of the final captions from the past 60 seconds for recovery. Each client's browser also keeps its own local store of captions. Finally the captions will be delivered to the live or recall view in the webpage.

All application state is held in memory. Restarting the server clears its revision state and recovery history, and closing a browser tab clears that browser's local store.

1.1 Server Architecture

When a connection opens, the server waits up to 5 seconds for its first "HELLO" message and uses it to decide whether a caption source or client has connected and the respective handler handles the connection (`handle_source()/handle_client()`).

The server stores these states that are important to its operation (Refer to Figure 1.1 in appendix):

- `CaptionRevisionState`**:** stores the info of the latest captions the server has accepted. It is used to perform deduplication and reject any stale (older) provisional captions.
- `FinalizedHistory`**:** stores the final captions, from the last 60 seconds only. Older captions are removed automatically. It is used to send missed finals to a client that reconnects or joins late.
- `CLIENTS`**:** holds the currently connected clients, each with its own `ClientState`. Every accepted caption update is offered to each connected client subscribed to the matching session (i.e., a pub/sub system).
- `ClientState`**:** one per connected client. It holds a queue of at most 64 pending messages, a sender task that sends them out in order, a lag monitor that checks whether the client is falling behind, and a small holding area for the newest provisional caption of each segment. The holding area is only used while the client is lagging (Section 3.2.3).

A caption update moves through the server in the following manner (refer to Sequence Diagram 1.2 in appendix):

1. The source handler receives the update and checks it against the revision tracker (using `CaptionRevisionState`).
2. If the update is final, a copy is stored in the recovery history (in `FinalizedHistory`).
3. The update is offered to every client registered in `CLIENTS`. Normally it joins that client's queue, but a lagging client or a full queue is handled differently (Section 3.2).
4. Each client's sender task then takes the messages from that client's own queue and sends them to the client over its WebSocket connection.

When a client connects or reconnects, the server:

1. Builds the missing history (`FinalizedHistory`, Section 2.2).
2. Creates the client's state `ClientState` and puts the missing history in its queue. It also starts the client's sender and lag monitor, and adds it to `CLIENTS` so it receives live caption updates.
3. When the connection ends, removes the client from `CLIENTS`, stops its tasks and clears its queue.

These mechanisms support deduplication, concurrent delivery, slow-client isolation, and semantic overload handling, which Section 3 describes in detail.

1.2 Client Architecture

The browser client is made up of two files (`client/index.html`, `client/app.js`) , which is just plain HTML and JavaScript. All of its state is held in the page's memory.

The client stores these states that are important to its operation (Refer to Figure 1.3) :

- `captions`: the caption store. It holds one entry per segment, which is the newest valid revision received for that segment. Captions older than 120 seconds are removed, which keeps memory bounded while still covering the 60-second recall window.
- `lastFinalSegment`, `lastFinalRevision`: the finalized cursor, which is the latest final caption the client has accepted. It is sent in every client's "HELLO".
- `liveRows`: links each segment to the line that displays it in the webpage's live view. When a newer revision of a segment arrives, the client updates that same line instead of adding a new one.
- `retryDelay`: how long to wait before the next reconnect attempt.

A caption update moves through the client in the following manner:

1. The client receives a live caption update (`CAPTION_UPDATE`), or a history message (`HISTORY`) from the server.
2. Each caption is checked before it is stored, whether it arrived live or in the history (`mergeCaption()`, Section 2.1).
3. If the caption is a newer final, the finalized cursor moves forward.
4. The page is redrawn from the caption store (`render()`). Provisional captions are shown in grey italics and finals in normal text in the browser. The "What did I miss?" panel filters the caption store for finals from the last 15/30/60 seconds, and thus it does not need to ask the server.

When the client connects or reconnects, it:

1. Opens the WebSocket to the server (`connect()`). It shows "Connected" once the socket opens and sends a client "HELLO" (`CLIENT_HELLO`) with its finalized cursor, asking for 60 seconds of history.
2. Receives the missing history first, and then live caption updates. Both go through the steps above.
3. When the connection drops, it shows "Disconnected, reconnecting in Ns" and tries again after `retryDelay` (`scheduleReconnect()`, Section 2.2).

For the demo, the webpage also has a "Simulate network drop (7s)" button. It closes the client's connection on purpose for 7 seconds, then reconnects.

1.3 Protocol

The source, server and clients talk to each other by sending JSON messages over WebSocket. The first message on a connection tells the server who has connected: the caption source or a student client. We kept the required message format exactly as given and only added extra checks and close codes (a number the server sends when it closes a WebSocket connection). We added the close codes as follows:

- **Handshake:** a connection with no valid "HELLO" within 5s is closed (code **1008**, "policy violation").
- **Dropping a client:** the server closes it with code **1013** ("try again later"), and the client reconnects to recover from HISTORY (Section 3.2).

Refer to Table 1.3 for more protocol details.

2 Revision Handling and Recovery

The server tracks the newest segment, revision, and whether it is final. It only accepts an update that is newer than the one stored: either a later segment, or a higher revision of the current segment that is not yet final. If a disconnect occurs halfway or the client connects after the source has started to send captions, captions that are missing can also be recovered.

2.1 Validation and Caption Construction

The server does most of the validation before broadcasting to the client's queue. On the server side, each caption update is validated using the source's session, segment and revision numbers, a boolean indicating if the segment is final and its timestamp. It rejects an older segment, a duplicate or lower revision of the current segment, and any update after that segment is final. If a segment passes all the validation, it is allowed to be broadcasted.

On the client side, each received segment is compared with the caption currently being held for that segment. The client ignores a lower revision, a provisional after a final, and most same-revision duplicates. It accepts a final segment replacing a provisional. It also rejects a provisional for a segment at or below its last-finalized cursor (refer to Section 1.2) if that segment is no longer stored. After the client validates a segment, the new caption content is added to a caption store containing all valid received captions. The client then updates the browser to display the change.

2.2 Recovery

The server keeps final captions from the last 60 seconds. Every time the client sends its “HELLO” handshake to the server, it also sends the latest finalized segment and revision received. Therefore, the server will know from which segment onwards to start sending the client data from its history store of captions. This is useful for clients that join late, or disconnect halfway, as the client can always request for segments that it did not receive, up to the last 60 seconds. By default, a client will request for the past 60 seconds of caption history when it connects. If a client disconnects, it automatically tries to reconnect by sending a “HELLO” handshake, with an initial delay of 1 second that doubles up to 10 seconds after each failed reconnect. When the connection is restored, the server replies the “HELLO” with the missed final caption history from the last received segment onwards.

3 Backpressure and Overload Handling

When a client cannot keep up with the captions being sent, the server should reduce that client’s load so that its final captions still arrive, without slowing down anyone else.

3.1 Preventing Inter-Client Head-of-Line (HOL) Blocking

In the starter template’s server, the source handler waits for `ws.send()` to finish for every client before it reads the next update. One slow client therefore holds up the source and every other client. Our server separates receiving from sending (Section 1.1). `broadcast()` only adds the update to each client’s own queue in `ClientState`. Adding to a queue takes almost no time, so `broadcast()` returns immediately and each client’s sender task delivers the update in the background. Only that client’s sender task writes to its WebSocket, so a slow client delays only its own queue while the source and the other clients carry on. Because each queue is bounded (Section 3.2.2), a slow client also cannot make the server’s memory grow without limit.

3.2 Identifying and Handling Slow Clients

Now that each client has their own queue to receive segments from, we needed a way to determine which clients were slow, and how to give these slower clients the highest chance of getting the final captions. We handle slow clients in three ways: a send timeout (Section 3.2.1), bounded queues with coalescing (Section 3.2.2), and lag probing (Section 3.2.3).

3.2.1 Send Timeout

Each send has a 10 second timeout. ws.send() only waits when the socket buffers are full, so if a client accepts no data for 10 seconds, the server closes the connection (code 1013). The client then reconnects and recovers its missed finals from the history (Section 2.2). This stops a stalled client from holding server resources.

3.2.2 Queues and Coalescing

The server maintains a pending queue of at most 64 messages for each connected client. When a new message arrives and the queue is full, the server compacts the client’s queue, removing provisionals that are older than the newest available for that segment, or removing all provisionals for a segment if a final already exists. This allows more final captions to enter the queue.

After compacting the queue, if the new message still cannot be added, the server closes the connection, causing the client to attempt a reconnection. Since the client starts a connection by asking for the caption history, the server uses that as a last opportunity to send final captions to the client.

3.2.3 Lag Probing

Sometimes the queue is not full, but the client is still slow. The sender task removes a message from the `ClientState` queue as soon as it hands it to the WebSocket connection (`ws.send()`). From there it can still wait in the kernel’s TCP send buffer, the network, and the client’s receive buffer and WebSocket queue. Therefore, an empty queue does not mean there is no backlog.

To find clients with this hidden backlog, the server sends a ping to each client every 0.5 seconds and measures how long the pong takes to come back, which is the round-trip time (RTT). The ping travels in the same TCP stream as the captions, so it has to wait behind any backlog further down the line, and its RTT shows how far behind the client really is. If a pong is late and still has not arrived 10 seconds after that, the server treats the client as stalled and closes the connection. The client then reconnects and recovers its missed finals through the “HELLO” request (Section 2.2).

The server remembers the best RTT for each client. A client is marked as lagging for the next 60 seconds if either of these happens:

- **Sudden lag:** one pong is more than 1.0s slower than the client’s best RTT. This catches a client that has fallen far behind, such as a slow reader.

- **Sustained lag:** the pong is more than 0.4s slower than the client's best RTT for 3 probes in a row. This catches steady congestion, such as a shared 2 Mbit/s link with 300 clients, where every client is a few hundred milliseconds late but none of them is over 1.0s slower.

While a client is lagging, finals are still queued straight away, but provisionals are not queued at all. Only the newest provisional for each segment is held aside, and it is thrown away once that segment's final arrives. The server restarts the 60 second timer every time it detects that the client is lagging. When 60 seconds pass with no lag, the held provisionals are sent and the client goes back to receiving every revision. Refer to Figure 3.2.

4 Public Experiments and Analysis

All tests were run on Ubuntu 25.04 VM (1 vCPU, 1.6GB RAM, Python 3.14.4, websockets 16.1.1) over loopback, using the unmodified instructor harness. Each test was run three times (`--runs 3`) using the commands in Table 4.1, and the full outputs with medians are in Tables 8.1 to 8.6. `delay_loss.sh` adds about 120 ms delay in each direction with jitter and 3% packet loss. To view the entire results in detail, please look at Table 8.1 to 8.6 in the appendix.

Metrics. Caption age (P50, P95, P99; clients' receive time minus `source_ts`); fresh ratio (age ≤500 ms, for all messages and for finals only); final completeness; normal update coverage; duplicate and stale counts; and normal against slow clients.

Analysis. Median completeness was 100% in every test, and duplicate and stale messages and connection errors were 0 in every run. Finals are never dropped from the queue : a final that cannot be sent immediately waits in the client's bounded queue, and a client that falls too far behind is evicted and recovers from the 60s history.

- **T3 (slow readers).** Slow clients received every final but sat seconds behind (median age 2876.68 ms, 30.69% fresh), while normal clients stayed at P95 14.37 ms, the same as without slow readers (T2: 14.49 ms), and 100% fresh. The slow clients' delay already sits in socket and TCP buffers, which the server cannot recall, so it detects lag from slow pongs and sends only finals.
- **T4 (reconnect).** The 20 disconnecting clients recovered 120 finals from `HISTORY`, giving 100% completeness. Final fresh is 98.42% because those 120 finals (1.58% of 7600) are old on arrival, and coverage is 98.33% because only finals are replayed.
- **T5 (delay and loss).** The tail (P95 616.11 ms, P99 898.97 ms) comes from how TCP handles packet loss. TCP resends lost packets, so completeness stays at 100%, but it delivers data in order, so later captions on the same connection wait behind the lost one (head-of-line blocking). A resend triggered by a timeout takes at least ~200 ms on Linux, and any resend costs at least one round trip of ~240 ms (2 * ~120 ms injected delay). Captions already take ~258 ms because of the injected delay, so one resend can push a caption to or past the 500 ms freshness limit, which lowers freshness to 92.41%. This is why P95 rises to 616.11 ms while the median stays at ~258 ms. Repeated losses likely explain the P99, which is ~2 resends later than the median. The starter server shows the same tail (P95 693.46 ms, Table 8.5), so it is network behaviour, not server queueing.
- **T6 (combined).** Normal clients stayed network-bounded (P95 713.28 ms, 87.91% fresh) and slow clients were 22.00% fresh. Coverage was lowest (85.26%) because slow clients receive finals only and some healthy clients are briefly flagged.
- **Recovery.** After an impairment ended, normal-client age returned to baseline and slow clients began receiving updates within 500 ms again (fresh) once their 60s hold expired. Normal coverage settled at 94.8% (Section 5.2, Table 5.2), because flagged clients wait out the 60s hold.

5 Before and After Optimization Comparison

5.1 Starter vs Final Server Comparison

Table 5.1 compares the starter server with our final server on T3 (10 slow clients) and T6 (20 slow clients plus delay and loss), using the median of three runs from Tables 8.3 and 8.6.

Final completeness rose from 95.13% to 100.00% on T3 and from 89.99% to 100.00% on T6, because the final server never drops a final. A final that cannot be sent immediately waits in the client's bounded queue, and a client that falls too far behind recovers from history on reconnection through eviction.

Slow mode, which sends finals only and holds the newest provisional per segment, cut tail latency (T3 P95 from 2268.67 ms to 17.56 ms, and T6 P99 from 26816.22 ms to 8613.75 ms). It also raised the slow-client fresh ratio from 1.69% to 30.69% on T3 and from 0.86% to 22.00% on T6, without hurting normal clients (P95 11.64 ms to 14.37 ms on T3, and 741.31 ms to 713.28 ms on T6).

The cost is lower than normal update coverage (from 95.18% to 94.43% on T3, and 90.15% to 85.26% on T6), since slow mode skips some provisional updates for flagged clients. However, we believe this decrease is a modest cost.

5.2 Lag Detection Margin

A client counts as lagging when its pong is slower than its own best round trip plus a margin. The baseline margin was 0.25s and the final was 1.0 s. Both were run on the same recovery workload (100 clients, 10% slow, delay and loss from 10s to 25s

of a 60s run, one run each). Completeness was 100% in both. Please refer to Table 5.2 for the breakdown.

Under 3% loss, a TCP retransmission can add a few hundred milliseconds to a pong, which exceeds the old 0.25s margin. Healthy clients were flagged repeatedly, and each flag cost a 60s finals-only hold. The 1.0s margin reduces this at the cost of slower detection of genuinely slow readers.

5.3 Escalating Hold policy

The baseline hold started at 10s and doubled to 60s on repeated flags, and the final holds at a flat 60s. With slow readers, the escalating hold lowered completeness to 97.8 - 98.3% at 120s or longer and slow-client freshness to 1.7% in the recovery test, against 32.53% for the flat hold (Table 5.2), which kept median completeness at 100% in every public test (Section 4). Each early release let provisionals refill the slow reader's socket buffers before the next pong exposed the backlog, so later finals queued behind them. The cost of the flat hold is slower convergence after a false flag.

6 Failures, Trade-offs and Limitations

6.1 Failures

- **False flags.** A delayed pong under loss or jitter can flag a healthy client, which then gets finals only for up to 60 s. This contributes to coverage of 96.32% in T5 and 85.26% in T6.
- **Late detection.** A slow reader's backlog is already in kernel buffers when its pong is delayed, so slow readers stay seconds behind even though they receive every final.
- **Constrained bandwidth.** When the shared link cannot carry even the finals-only stream in time, freshness cannot be recovered. At 500 clients on a 2Mbit link, median freshness was ~0.33% and coverage ~52.63%. Completeness was still 100% in two of three runs and 99.89% in the third.
- **Eviction.** A client whose queue overflows with finals is closed and must reconnect. If it misses the 60s history window, older finals are lost.

6.2 Trade-offs

We give up provisional coverage to guarantee every final and keep healthy clients fresh. A larger margin avoids false flags but detects slow readers later, and a longer fixed hold keeps long runs stable but converges slower. Per-client queues and probes add a small constant latency (T2 P99 21.94 ms) in exchange for isolating slow readers.

6.3 Limitations

Tests used one single-vCPU VM, shared by the server and the load clients, over loopback with tc impairment (20 to 300 clients, up to 300s), not a real network. The server handles one lecture session and keeps finalized history in memory only, for 60s. The thresholds (1.0s margin, 60s hold, 64-item queue) were tuned on the public workloads.

7 Future Work

Gradual recovery. After a hold ends, the server releases the held provisionals and immediately returns to sending every revision, which can refill a slow reader's buffers. A recovery phase would send one provisional every ~2s and return to full updates only after the client stays healthy. This could give faster convergence without the completeness loss seen with the escalating hold in Section 5.3.

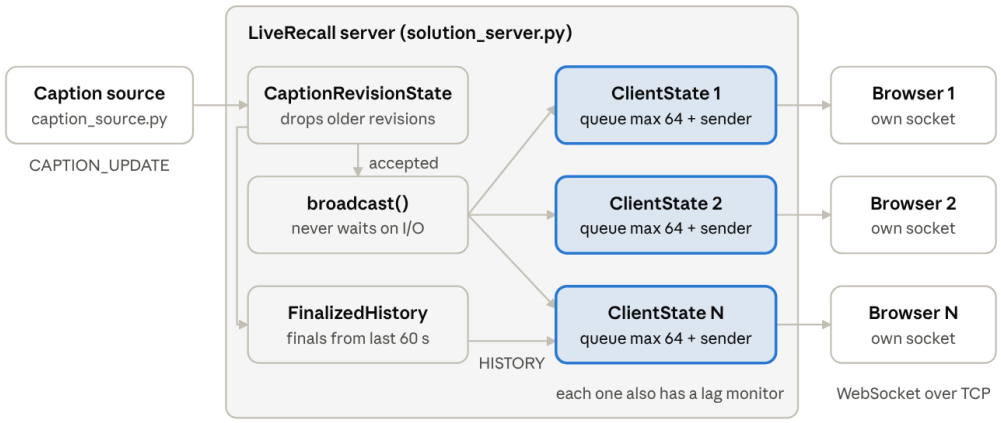
Adaptive hold length. The 60s hold is fixed. The hold could instead be sized from each client's measured drain rate, so a client that recovers quickly is released sooner and a persistently slow one stays held.

Direct backlog measurement. Lag is currently inferred from ping/pong delay, which retransmissions under packet loss can distort. Linux exposes the unsent socket bytes directly (`SIOCOUTQ`, `TCP_NOTSENT_LOWAT`). Reading them would remove the false flags at the source and is a suitable FYP direction.

Transport-level prioritisation. Provisionals and finals share one ordered TCP stream, so a stale provisional can delay a final. Per-stream priorities or unreliable datagrams for provisionals (WebTransport/QUIC) would let finals bypass them.

Appendices

Figure 1.1 - High level server architecture



Sequence Diagram 1.2 - Caption flow

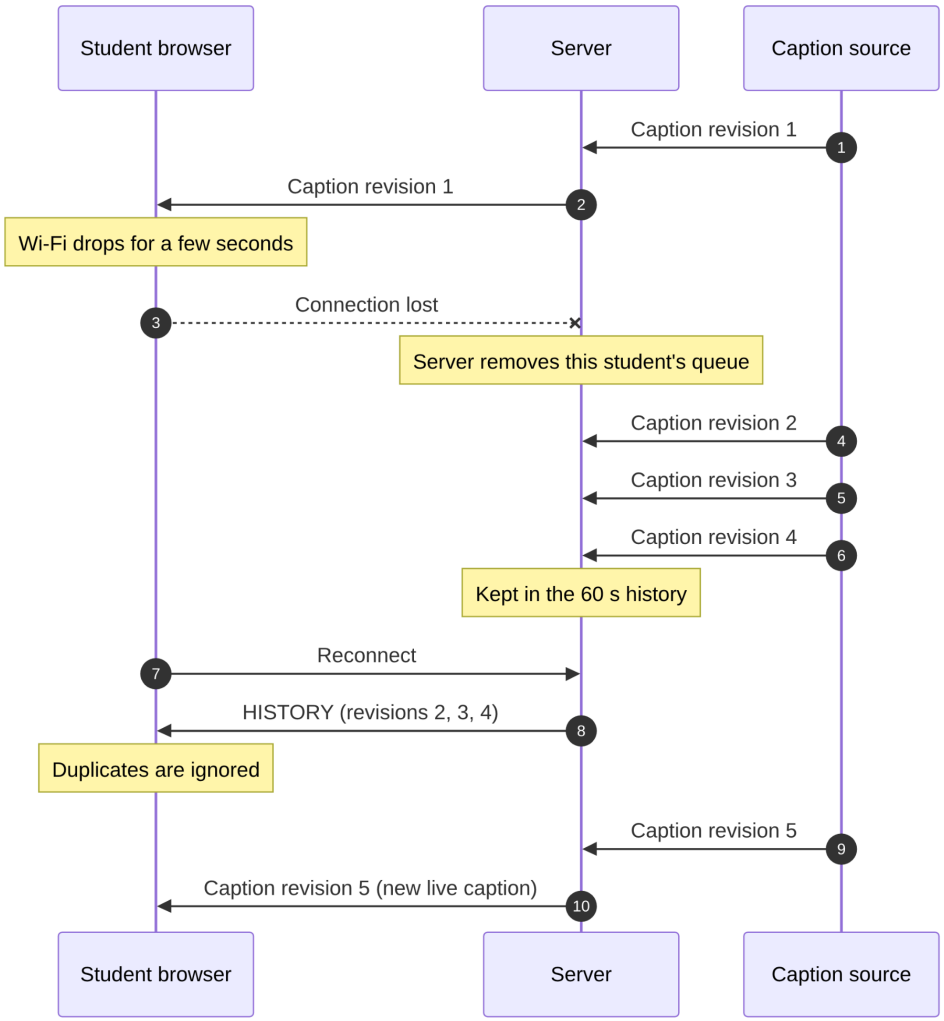


Figure 1.3 - Client architecture

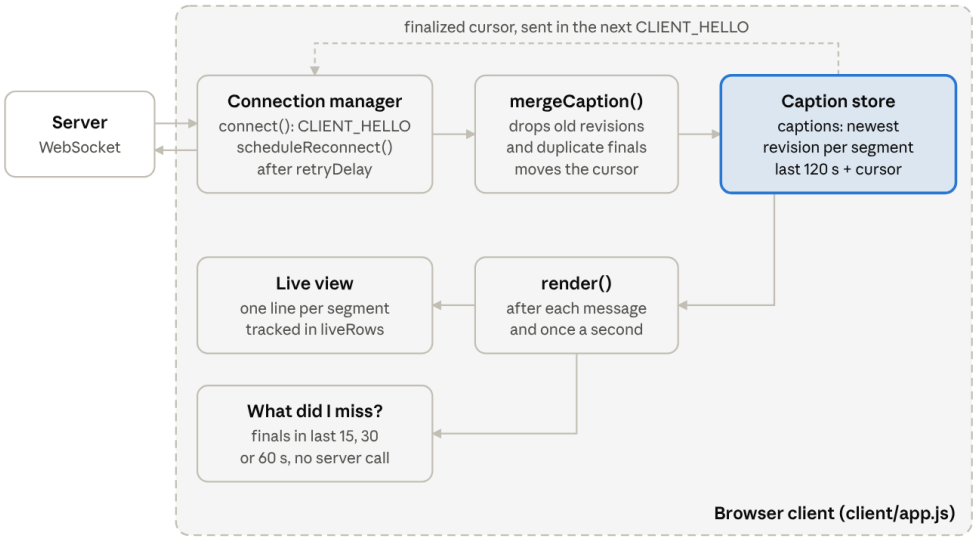


Table 1.3

Message	Sent from → to	Main fields	What the server does
`SOURCE_HELLO`	Source → Server	`session`	Registers the caption source.
`CAPTION_UPDATE`	Source → Server → Clients	`session, segment, revision, source_ts, final, text`	Checks the update (Section 2.1), then forwards it to clients unchanged. Finals are also saved in the history.
`SOURCE_DONE`	Source → Server	none	Logged only. Not sent to clients.
`CLIENT_HELLO`	Client → Server	`client_id, session, last_segment, last_revision, history_seconds`	Tells the server the last final the client has and how much history it wants (Section 2.2).
`HISTORY`	Server → Client	`items` (a list of finals)	The first message every client receives. It holds only the finals the client has not seen yet, oldest first.

Table 3.1 - Classes Table

Host	Handler	State Class
Source	handle_source()	FinalizedHistory and CaptionRevisionState
Client	handle_client()	ClientState

Figure 3.2 - Slow-client state diagram

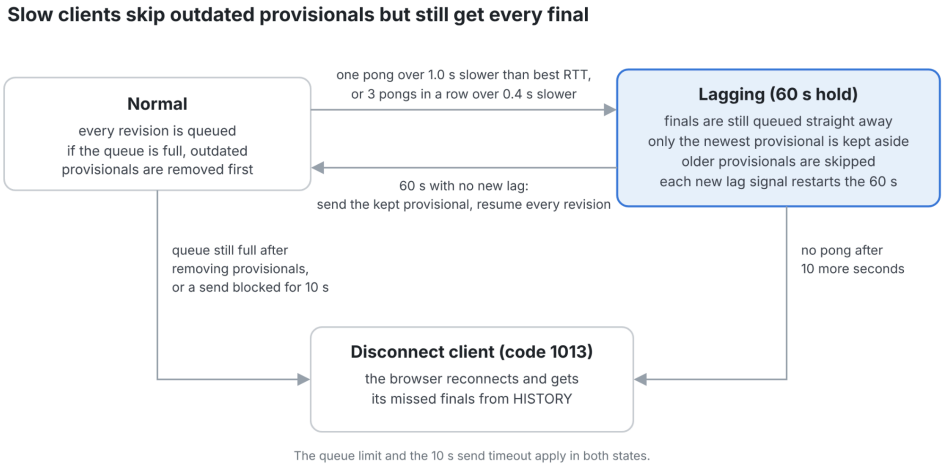


Table 4.1 - Public Test Cases

```python
## T1 - Basic Correctness
python run_demo.py --clients 20 --duration 60 --runs 3

## T2 - Client Scaling
python run_demo.py --clients 100 --duration 60 --runs 3

## T3 - Slow Clients
python run_demo.py --clients 100 --duration 60 --runs 3 --slow-percent 10 --slow-ms 500

## T4 - Disconnect Reconnect
python run_demo.py --clients 100 --duration 60 --runs 3 --disconnect-percent 20 --disconnect-at 15

## T5 - Network Impairment
sudo ./network_profiles/delay_loss.sh lo
python run_demo.py --clients 100 --duration 60 --runs 3
sudo ./network_profiles/reset.sh lo

## T6 - T3 (Slow Clients) + T5 (Network Impairment)
sudo ./network_profiles/delay_loss.sh lo
python run_demo.py --clients 100 --duration 60 --runs 3 --slow-percent 20 --slow-ms 500
sudo ./network_profiles/reset.sh lo
```

Table 5.1 - Starter vs Final Server Comparison (Median of 3 runs)

Metric	T3 Starter	T3 Final	T6 Starter	T6 Final
P50 caption age	9.11 ms	10.39 ms	260.15 ms	267.85 ms
P95 caption age	2268.67 ms	17.56 ms	16611.16 ms	3700.85 ms
P99 caption age	23975.29 ms	6572.77 ms	26816.22 ms	8613.75 ms
Fresh caption ratio	94.65%	96.75%	78.91%	80.87%
Fresh final ratio	90.00%	93.82%	70.26%	73.76%
Final completeness	95.13%	100.00%	89.99%	100.00%
Normal update coverage	95.18%	94.43%	90.15%	85.26%
Normal-client P95	11.64 ms	14.37 ms	741.31 ms	713.28 ms
Slow-client P95	28017.74 ms	8571.64 ms	27993.64 ms	9268.39 ms

Slow-client fresh ratio	1.69%	30.69%	0.86%	22.00%

Table 5.2 - Lag Margin

Metric	0.25s	1.0s
Overall coverage	45.16%	90.95%
Normal coverage, 25-60s	33.3%	94.8%
Overall P95	2038ms	512ms
Slow fresh	35.35%	32.53%

Table 8.1 - Comparison on Basic Correctness

Clients=20, duration=60.0s, source segments=76
Starting 20 clients; slow=0, disconnecting=0, late=0
SOURCE_DONE: emitted 76 finalized segments

Category	Original			Our Solution		
Clients observed	20	20	20	20	20	20
Live caption messages	4560	4560	4560	4560	4560	4560
Recovered history items	0	0	0	0	0	0
Accepted messages	4560	4560	4560	4560	4560	4560
Recovered messages	0	0	0	0	0	0
Duplicate messages observed	0	0	0	0	0	0
Stale messages observed	0	0	0	0	0	0
P50 caption age	2.59 ms	2.56 ms	2.58 ms	2.86 ms	3.00 ms	2.91 ms
P95 caption age	4.45 ms	4.03 ms	4.33 ms	4.53 ms	4.82 ms	4.84 ms
P99 caption age	6.21 ms	5.49 ms	7.68 ms	5.71 ms	5.87 ms	6.78 ms
Fresh Caption Ratio (<=500 ms)	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Fresh Final Caption Ratio (<=500 ms)	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Successful connections	20	20	20	20	20	20
Connection errors	0	0	0	0	0	0
Unique final captions	1520/1520	1520/1520	1520/1520	1520/1520	1520/1520	1520/1520
Final-caption completeness	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Unique live revisions	4560/4560	4560/4560	4560/4560	4560/4560	4560/4560	4560/4560
Normal Update Coverage	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%

=== Median across independent runs ===	
P50 caption age	2.58 ms
P95 caption age	4.33 ms
P99 caption age	6.21 ms
Fresh Caption Ratio	100.00 %
Fresh Final Caption Ratio	100.00 %
Final completeness	100.00 %
Normal Update Coverage	100.00 %
Normal-client P95	4.33 ms

=== Median across independent runs ===	
P50 caption age	2.91 ms
P95 caption age	4.82 ms
P99 caption age	5.87 ms
Fresh Caption Ratio	100.00 %
Fresh Final Caption Ratio	100.00 %
Final completeness	100.00 %
Normal Update Coverage	100.00 %
Normal-client P95	4.82 ms

Table 8.2 - Comparison on Client Scaling

Clients=100, duration=60.0s, source segments=76
Starting 100 clients; slow=0, disconnecting=0, late=0
SOURCE_DONE: emitted 76 finalized segments

Category	Original			Our Solution		
Clients observed	100	100	100	100	100	100
Live caption messages	22800	22800	22800	22800	22800	22800
Recovered history items	0	0	0	0	0	0
Accepted messages	22800	22800	22800	22800	22800	22800
Recovered messages	0	0	0	0	0	0
Duplicate messages observed	0	0	0	0	0	0
Stale messages observed	0	0	0	0	0	0
P50 caption age	9.54 ms	9.24 ms	9.15 ms	10.84 ms	9.94 ms	9.47 ms
P95 caption age	12.46 ms	11.46 ms	11.19 ms	14.49 ms	13.54 ms	17.60 ms
P99 caption age	16.24 ms	12.59 ms	11.81 ms	21.94 ms	17.21 ms	33.44 ms
Fresh Caption Ratio (<=500 ms)	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Fresh Final Caption Ratio (<=500 ms)	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Successful connections	100	100	100	100	100	100
Connection errors	0	0	0	0	0	0
Unique final captions	7600/7600	7600/7600	7600/7600	7600/7600	7600/7600	7600/7600
Final-caption completeness	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Unique live revisions	22800/22800	22800/22800	22800/22800	22800/22800	22800/22800	22800/22800
Normal Update Coverage	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%

=== Median across independent runs ===	
P50 caption age	9.24 ms
P95 caption age	11.46 ms
P99 caption age	12.59 ms
Fresh Caption Ratio	100.00 %
Fresh Final Caption Ratio	100.00 %
Final completeness	100.00 %
Normal Update Coverage	100.00 %
Normal-client P95	11.46 ms

=== Median across independent runs ===	
P50 caption age	9.94 ms
P95 caption age	14.49 ms
P99 caption age	21.94 ms
Fresh Caption Ratio	100.00 %
Fresh Final Caption Ratio	100.00 %
Final completeness	100.00 %
Normal Update Coverage	100.00 %
Normal-client P95	14.49 ms

Table 8.3 - Comparison on Slow Clients

Clients=100, duration=60.0s, source segments=76
Starting 100 clients; slow=10, disconnecting=0, late=0
SOURCE_DONE: emitted 76 finalized segments

Category	Original			Our Solution		
Clients observed	100	100	100	100	100	100
Live caption messages	21700	21700	21700	21530	21536	21530
Recovered history items	0	0	0	0	0	0
Accepted messages	21700	21700	21700	21530	21536	21530
Recovered messages	0	0	0	0	0	0
Duplicate messages observed	0	0	0	0	0	0
Stale messages observed	0	0	0	0	0	0
P50 caption age	9.11 ms	9.75 ms	8.86 ms	10.48 ms	10.39 ms	10.34 ms
P95 caption age	2243.87 ms	2269.65 ms	2268.67 ms	17.56 ms	15.33 ms	18.29 ms
P99 caption age	23975.29 ms	23954.62 ms	24018.48 ms	6572.77 ms	6755.59 ms	6571.65 ms
Fresh Caption Ratio (<=500 ms)	94.65%	94.65%	94.65%	96.80%	96.69%	96.75%
Fresh Final Caption Ratio (<=500 ms)	90.00%	90.00%	90.00%	93.95%	93.74%	93.82%
Successful connections	100	100	100	100	100	100
Connection errors	0	0	0	0	0	0
Unique final captions	7230/7600	7230/7600	7230/7600	7600/7600	7600/7600	7600/7600
Final-caption completeness	95.13%	95.13%	95.13%	100.00%	100.00%	100.00%
Unique live revisions	21700/22800	21700/22800	21700/22800	21530/22800	21536/22800	21530/22800
Normal Update Coverage	95.18%	95.18%	95.18%	94.43%	94.46%	94.43%

Slow-reader isolation

Normal clients: P50=9.03 ms, P95=11.64 ms, Fresh=100.00% (20520 msgs)
Slow clients: P50=14694.92 ms, P95=28017.74 ms, Fresh=1.69% (1180 msgs)

Normal clients: P50=9.65 ms, P95=11.70 ms, Fresh=100.00% (20520 msgs)
Slow clients: P50=14686.40 ms, P95=27981.39 ms, Fresh=1.69% (1180 msgs)

Normal clients: P50=8.77 ms, P95=10.70 ms, Fresh=100.00% (20520 msgs)
Slow clients: P50=14718.44 ms, P95=28060.22 ms, Fresh=1.69% (1180 msgs)

=== Median across independent runs ===

P50 caption age	9.11 ms
P95 caption age	2268.67 ms
P99 caption age	23975.29 ms
Fresh Caption Ratio	94.65 %
Fresh Final Caption Ratio	90.00 %
Final completeness	95.13 %
Normal Update Coverage	95.18 %
Normal-client P95	11.64 ms
Slow-client P95	28017.74 ms

Slow-reader isolation

Normal clients: P50=10.37 ms, P95=14.98 ms, Fresh=100.00% (20520 msgs)
Slow clients: P50=2876.68 ms, P95=8571.64 ms, Fresh=31.68% (1010 msgs)

Normal clients: P50=10.30 ms, P95=13.64 ms, Fresh=100.00% (20520 msgs)
Slow clients: P50=3015.23 ms, P95=8852.20 ms, Fresh=29.92% (1016 msgs)

Normal clients: P50=10.22 ms, P95=14.37 ms, Fresh=100.00% (20520 msgs)
Slow clients: P50=2855.50 ms, P95=8556.76 ms, Fresh=30.69% (1010 msgs)

=== Median across independent runs ===

P50 caption age	10.39 ms
P95 caption age	17.56 ms
P99 caption age	6572.77 ms
Fresh Caption Ratio	96.75 %
Fresh Final Caption Ratio	93.82 %
Final completeness	100.00 %
Normal Update Coverage	94.43 %
Normal-client P95	14.37 ms
Slow-client P95	8571.64 ms

Table 8.4 - Comparison on Disconnect Reconnect

Clients=100, duration=60.0s, source segments=76
Starting 100 clients; slow=0, disconnecting=20, late=0
SOURCE_DONE: emitted 76 finalized segments

Category	Original			Our Solution		
Clients observed	100	100	100	100	100	100
Live caption messages	22420	22420	22420	22420	22420	22420
Recovered history items	0	0	0	120	120	120
Accepted messages	22420	22420	22420	22420	22420	22420
Recovered messages	0	0	0	120	120	120
Duplicate messages observed	0	0	0	0	0	0
Stale messages observed	0	0	0	0	0	0
P50 caption age	7.59 ms	7.88 ms	7.80 ms	9.27 ms	9.42 ms	9.06 ms
P95 caption age	10.47 ms	11.39 ms	11.36 ms	13.08 ms	13.17 ms	12.58 ms
P99 caption age	12.36 ms	13.00 ms	16.28 ms	15.56 ms	16.36 ms	14.78 ms
Fresh Caption Ratio (<=500 ms)	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Fresh Final Caption Ratio (<=500 ms)	98.42%	98.42%	98.42%	98.43%	98.42%	98.42%
Successful connections	120	120	120	120	120	120
Connection errors	0	0	0	0	0	0
Unique final captions	7480/7600	7480/7600	7480/7600	7600/7600	7600/7600	7600/7600
Final-caption completeness	98.42%	98.42%	98.42%	100.00%	100.00%	100.00%
Unique live revisions	22420/22800	22420/22800	22420/22800	22420/22800	22420/22800	22420/22800
Normal Update Coverage	98.33%	98.33%	98.33%	98.33%	98.33%	98.33%

Reconnect / late-join HISTORY response

Recovery P50: 2.25 ms
Recovery P95: 2.64 ms

Recovery P50: 11.24 ms
Recovery P95: 19.15 ms

Recovery P50: 7.50 ms
Recovery P95: 8.05 ms

Note: fast HISTORY response alone does not prove correct recovery; check final completeness and duplicate/stale counts too.

=== Median across independent runs ===

P50 caption age	7.80 ms
P95 caption age	11.36 ms
P99 caption age	13.00 ms
Fresh Caption Ratio	100.00 %
Fresh Final Caption Ratio	98.42 %
Final completeness	98.42 %
Normal Update Coverage	98.33 %
Normal-client P95	11.36 ms
Recovery P95	8.05 ms

Reconnect / late-join HISTORY response

Recovery P50: 6.44 ms
Recovery P95: 7.83 ms

Recovery P50: 6.63 ms
Recovery P95: 6.71 ms

Recovery P50: 9.74 ms
Recovery P95: 10.38 ms

=== Median across independent runs ===

P50 caption age	9.27 ms
P95 caption age	13.08 ms
P99 caption age	15.56 ms
Fresh Caption Ratio	100.00 %
Fresh Final Caption Ratio	98.42 %
Final completeness	100.00 %
Normal Update Coverage	98.33 %
Normal-client P95	13.08 ms
Recovery P95	7.83 ms

Table 8.5 - Comparison on Network Impairment

Clients=100, duration=60.0s, source segments=76
Starting 100 clients; slow=0, disconnecting=0, late=0
SOURCE_DONE: emitted 76 finalized segments

Category	Original			Our Solution		
Clients observed	100	100	100	100	100	100
Live caption messages	22786	22799	22792	21222	21999	21962
Recovered history items	0	0	0	0	4	0
Accepted messages	22786	22799	22792	21222	21999	21962
Recovered messages	0	0	0	0	4	0
Duplicate messages observed	0	0	0	0	0	0
Stale messages observed	0	0	0	0	0	0
P50 caption age	260.82 ms	251.08 ms	258.42 ms	258.81 ms	257.71 ms	252.43 ms
P95 caption age	693.46 ms	615.44 ms	717.42 ms	568.50 ms	692.31 ms	616.11 ms
P99 caption age	1057.91 ms	877.58 ms	1034.02 ms	898.97 ms	1005.57 ms	832.23 ms
Fresh Caption Ratio (<=500 ms)	89.54%	93.23%	87.46%	93.02%	88.56%	92.41%
Fresh Final Caption Ratio (<=500 ms)	89.61%	93.24%	87.95%	93.26%	87.01%	91.14%
Successful connections	100	100	100	100	100	100
Connection errors	0	0	0	0	0	0
Unique final captions	7596/7600	7599/7600	7600/7600	7600/7600	7600/7600	7600/7600
Final-caption completeness				100.00%	100.00%	100.00%
Unique live revisions	99.95%	99.99%	100.00%	21222/22800	21999/22800	21962/22800
Normal Update Coverage	22786/22800	22799/22800	22792/22800	93.08%	96.49%	96.32%
	99.94%	100.00%	99.96%			

=== Median across independent runs ===	
P50 caption age	258.42 ms
P95 caption age	693.46 ms
P99 caption age	1034.02 ms
Fresh Caption Ratio	89.54 %
Fresh Final Caption Ratio	89.61 %
Final completeness	99.99 %
Normal Update Coverage	99.96 %
Normal-client P95	693.46 ms

=== Median across independent runs ===	
P50 caption age	257.71 ms
P95 caption age	616.11 ms
P99 caption age	898.97 ms
Fresh Caption Ratio	92.41 %
Fresh Final Caption Ratio	91.14 %
Final completeness	100.00 %
Normal Update Coverage	96.32 %
Normal-client P95	616.11 ms

Table 8.6 - Comparison on Slow Clients and Network Impairment

Clients=100, duration=60.0s, source segments=76
Starting 100 clients; slow=20, disconnecting=0, late=0
SOURCE_DONE: emitted 76 finalized segments

Category	Original			Our Solution		
Clients observed	100	100	100	100	100	100
Live caption messages	20572	20532	20554	19618	19439	19391
Recovered history items	0	0	0	0	2	0
Accepted messages	20572	20532	20554	19618	19439	19391
Recovered messages	0	0	0	0	2	0
Duplicate messages observed	0	0	0	0	0	0
Stale messages observed	0	0	0	0	0	0
P50 caption age	259.79 ms	260.15 ms	270.30 ms	271.76 ms	267.85 ms	257.81 ms
P95 caption age	16665.68 ms	16611.16 ms	16408.41 ms	3587.92 ms	3866.78 ms	3700.85 ms
P99 caption age	26876.76 ms	26816.22 ms	26652.42 ms	8613.75 ms	8787.94 ms	8610.74 ms
Fresh Caption Ratio (<=500 ms)	79.84%	77.45%	78.91%	79.01%	80.87%	84.32%
Fresh Final Caption Ratio (<=500 ms)	71.18%	68.34%	70.26%	73.76%	73.70%	79.21%
Successful connections	100	100	100	100	100	100
Connection errors	0	0	0	0	0	0
Unique final captions	6858/7600	6832/7600	6839/7600	7600/7600	7600/7600	7598/7600
Final-caption completeness	90.24%	89.89%	89.99%	100.00%	100.00%	99.97%
Unique live revisions	20572/22800	20532/22800	20554/22800	19618/22800	19439/22800	19391/22800
Normal Update Coverage	90.23%	90.05%	90.15%	86.04%	85.26%	85.05%

Slow-reader isolation

Normal clients: P50=251.15 ms, P95=699.52 ms, Fresh=89.96% (18234 msgs)
Slow clients: P50=14863.83 ms, P95=27993.64 ms, Fresh=0.86% (2338 msgs)

Normal clients: P50=251.04 ms, P95=786.63 ms, Fresh=87.31% (18215 msgs)
Slow clients: P50=14888.08 ms, P95=28126.84 ms, Fresh=0.00% (2317 msgs)

Normal clients: P50=261.36 ms, P95=741.31 ms, Fresh=88.85% (18234 msgs)
Slow clients: P50=14879.83 ms, P95=27947.62 ms, Fresh=0.86% (2320 msgs)

=== Median across independent runs ===

P50 caption age	260.15 ms
P95 caption age	16611.16 ms
P99 caption age	26816.22 ms
Fresh Caption Ratio	78.91 %
Fresh Final Caption Ratio	70.26 %
Final completeness	89.99 %
Normal Update Coverage	90.15 %
Normal-client P95	741.31 ms
Slow-client P95	27993.64 ms

Slow-reader isolation

Normal clients: P50=264.70 ms, P95=781.14 ms, Fresh=85.51% (17586 msgs)
Slow clients: P50=3322.13 ms, P95=9268.39 ms, Fresh=22.79% (2032 msgs)

Normal clients: P50=261.30 ms, P95=713.28 ms, Fresh=87.91% (17392 msgs)
Slow clients: P50=3547.86 ms, P95=9330.64 ms, Fresh=21.06% (2047 msgs)

Normal clients: P50=252.15 ms, P95=637.64 ms, Fresh=91.63% (17355 msgs)
Slow clients: P50=3294.32 ms, P95=9245.00 ms, Fresh=22.00% (2036 msgs)

=== Median across independent runs ===

P50 caption age	267.85 ms
P95 caption age	3700.85 ms
P99 caption age	8613.75 ms
Fresh Caption Ratio	80.87 %
Fresh Final Caption Ratio	73.76 %
Final completeness	100.00 %
Normal Update Coverage	85.26 %
Normal-client P95	713.28 ms
Slow-client P95	9268.39 ms